M.A.C.H: Multimodal Academic Cognitive Hub — A Fine-Tuned Graph-RAG Framework for Campus Knowledge Systems.

Dr. Shanmuga Priya S
Assistant Professor (Sl.Gd.), Department of CSE
ss_priya@cb.amrita.edu

A. Chandra Vamsi
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.sc.u4cse23505@cb.students.amrita.edu

B. Prem Sai Charan
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.sc.u4cse23508@cb.students.amrita.edu

M. Pavan Prem Prabhas
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.sc.u4cse23530@cb.students.amrita.edu

P. Tirupati Reddy
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.sc.u4cse23568@cb.students.amrita.edu

***Abstract*—University academic information is often difficult to access because syllabi are organized hierarchically, prerequisite relationships span multiple courses, and past-year question papers (PYQs) contain diagrams that can be lost during text extraction. General-purpose chatbots may generate answers based on their pretrained knowledge rather than official curriculum documents, while conventional vector search can retrieve information from unrelated courses that cover similar topics. To address these challenges, we present M.A.C.H. (Multimodal Academic Cognitive Hub) a framework developed for higher education curriculum systems. The framework combines four main components: (1) a Neo4j curriculum knowledge graph constructed using a boundary-preserving, regex-first syllabus parser, with LLM verification for ambiguous units; (2) a two-tier intent router that first applies keyword rules and then uses embedding similarity across intent descriptions to direct structured queries to graph traversals and open-ended queries to PostgreSQL hybrid search (combining pgvector HNSW and `tsvector` rankings through Reciprocal Rank Fusion); (3) a two-stage question-to-topic mapper that limits LLM decisions to retrieved candidate topics and abstains when confidence is insufficient; and (4) a geometry-based PYQ extraction method that uses PDF coordinates to crop questions, includes relevant vector drawings, and stitches figures that extend across pages. We also present a synthetic instruction dataset containing 9,000 examples for small language model task adaptation. Experimental evaluations on engineering curriculum data demonstrate that M.A.C.H. achieves 100.0% prerequisite traversal exact-match accuracy on the evaluated 10-course statutory DAG (with 87.40% course entity resolution across 254 natural-language queries), 92.7% intent routing accuracy (92.8% macro-F1) on template queries, 99.3% topic mapping accuracy with 99.3% auto-approval precision, and sub-120ms response latencies for structured curriculum queries via its Zero-LLM Fast-Path.**

I. Introduction

University academic information is spread across different types of documents and structures. Syllabi organize information into departments, courses, units, and topics, while prerequisite relationships connect courses. Exam papers also contain marks, course-outcome (CO) tags, Bloom's taxonomy levels, and diagrams that may be handwritten or typeset. Students often ask questions about this information in everyday language, such as "what do I study in year 1" or "what are the textbooks for it". However, serving these queries through a general-purpose chatbot or a conventional vector search system presents two main challenges.

- **Fragmentation and contamination.** Fixed-size chunking can separate topics from the units and courses they belong to, making it difficult to preserve their original context. In addition, a topic such as "Classification" may appear in several courses, with different meanings and applications. Without course-specific filtering, nearest-neighbour search may retrieve information from an unrelated course, resulting in irrelevant or misleading answers.
- **Lost visual content.** Exam papers often include circuit diagrams and equations that are essential to understanding a question. Plain-text extraction or OCR-based processing can omit these elements or represent them inaccurately.

M.A.C.H. addresses these challenges through a graph-first design that uses language models for tasks requiring language

understanding while relying on stored academic records as the source of curriculum information. The main contributions of this work are:

- a dual-persistence architecture that combines a Neo4j curriculum knowledge graph with PostgreSQL hybrid search, applying course-code filtering where supported (Sections III–V);
- a deterministic, boundary-preserving syllabus ingestion pipeline with selective LLM verification (Section IV);
- a two-tier intent router and a hybrid retrieval mechanism based on Reciprocal Rank Fusion (RRF) (Section V);
- a two-stage question-to-topic mapping method that restricts candidate selection and supports abstention when confidence is insufficient (Section VI);
- a layout-preserving PYQ extraction method that assigns deterministic identities to questions (Section VII);
- a synthetic instruction dataset containing 9,000 examples, with its diversity examined in this work (Section VIII); and
- empirical benchmarking demonstrating 100.0% prerequisite exact-match accuracy on the evaluated curriculum DAG, 87.40% course entity resolution, 100.0% deterministic question identity stability with 98.02% metadata coverage, and sub-120ms Zero-LLM Fast-Path retrieval (Section IX).

Organization. The remainder of this paper is organized as follows: Section III outlines the system architecture; Section IV details syllabus parsing and the Neo4j ontology; Section V explains intent classification and hybrid RRF search; Section VI specifies the two-stage topic mapper and Algorithm 1; Section VII presents layout-preserving PYQ extraction; Section VIII presents the 9,000-sample synthetic dataset; Section IX provides empirical benchmarks, latency profiling, and ablation analysis; Section X discusses architectural advantages and data governance; and Section XI concludes the paper.

II. Related Work

Retrieval-augmented generation. Retrieval-augmented generation (RAG) improves answer generation by providing a model with information retrieved from external sources [1]. Existing research examines several RAG approaches, ranging from basic retrieval pipelines to more advanced and modular systems [2]. Graph-based RAG uses knowledge graphs to organize information and answer questions by connecting related information [3]. M.A.C.H. follows a different approach by constructing its knowledge graph from official syllabi and answering structured queries through explicit Cypher traversals. Therefore, the term "Graph-RAG" in this work refers to knowledge-graph-grounded retrieval rather than the community-summary approach described in [3].

Hybrid retrieval. Dense and lexical retrieval methods have different strengths, so combining their rankings can improve retrieval without requiring their scores to be directly comparable. Reciprocal Rank Fusion (RRF) combines ranked results using the reciprocal of each result's rank, adjusted by a constant [4]. Hierarchical Navigable Small World (HNSW) graphs enable approximate nearest-neighbour search [5] and support the pgvector index used in M.A.C.H. The question-to-topic mapper uses BM25 for lexical ranking [6], whereas document-level retrieval relies on PostgreSQL's `ts_rank`. The system generates semantic embeddings using `nomic-embed-text` [7].

Educational knowledge and taxonomy. M.A.C.H. uses the six cognitive levels of the revised Bloom's taxonomy to annotate topics and questions [8]. This taxonomy builds upon the original framework [9]. Neo4j's graph traversal capabilities also allow the system to retrieve prerequisite relationships across multiple levels without relying on recursive relational joins.

Layout-aware extraction. Tools such as LayoutParser [10] demonstrate how spatial information can help preserve document structure during technical document processing. Rather than using a general-purpose layout-parsing framework, M.A.C.H. employs a lightweight, rule-based method tailored to typical exam-paper layouts. It works directly with PDF coordinates and vector drawings to preserve question content and its associated diagrams.

Efficient model adaptation. Techniques such as LoRA [11] and QLoRA [12] make language-model fine-tuning more practical by reducing computational and memory requirements. The Qwen2.5 model family [13] provides potential base models for future experiments. In M.A.C.H., the 9,000-example synthetic instruction dataset described in Section VIII is intended to support future fine-tuning of small language models. Fine-tuning and adaptation experiments have not yet begun. RAGAS [14] provides a potential approach for evaluating the quality of retrieval-augmented answers in future work.

III. System Architecture

M.A.C.H. is an asynchronous Python service (FastAPI, SQLAlchemy with asyncpg, Neo4j Bolt driver) with a zero-build vanilla-JavaScript progressive web application. Fig. 1 shows the components. Two stores serve different query types: Neo4j holds the curriculum graph for deterministic relational lookups, and PostgreSQL with pgvector holds chunk text, embeddings and a stored `tsvector` column for unstructured content. Language and vision models run in Ollama; in the current configuration a single 12B model, `gemma4:12b-it-qat`, serves both text generation and image description, and `nomic-embed-text` (768 dimensions) provides embeddings, cached in an in-memory LRU of 2,000 entries.

A. Request Lifecycle

When a student submits an inquiry, requests containing uploaded diagram images bypass text-only processing: the multimodal vision model directly classifies the image into a canonical curriculum topic, retrieving mapped past-year examination questions (PYQs) from the graph. For text inquiries, follow-up messages first pass through an LLM query rewriter

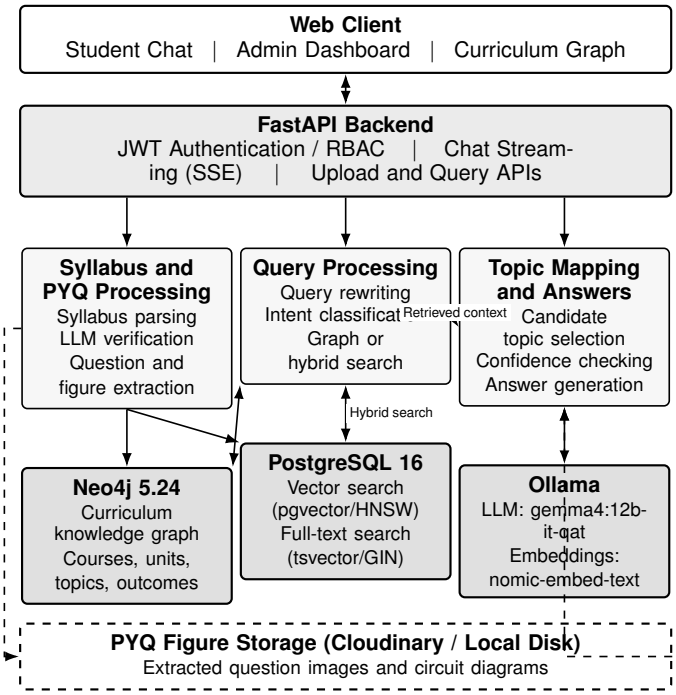


Fig. 1. Architecture of M.A.C.H. The FastAPI backend connects the web client to the syllabus-processing, query-processing, and answer-generation modules. Syllabus and examination data are stored in Neo4j, PostgreSQL, and the media store as appropriate. Queries use graph traversal or hybrid retrieval, while local Ollama models support language processing and embedding generation.

TABLE I
TECHNOLOGY STACK AS CONFIGURED IN THE REPOSITORY

Layer	Component and Setting	Role
API	FastAPI + Uvicorn (ASGI, Chunked Streaming)	Chat, upload, admin, graph endpoints; JWT with role checks
Graph store	Neo4j Community 5.24	Curriculum graph, prerequisite and question–topic edges
Relational + vector	PostgreSQL 16 + pgvector (HNSW $m = 16$, $ef = 64$)	Document chunks, 768-d embeddings, GIN-indexed `tsvector`
Embeddings	`nomic-embed-text` via Ollama (768-d, LRU cache of 2,000)	Query, chunk, and topic embeddings
Language / vision	`gemma4:12b-it-qat` via Ollama (12B, quantization-aware)	Generation, query rewriting, topic re-ranking, visual topic classification
PDF processing	PyMuPDF 1.24+, Pillow	Text blocks, bounding boxes, vector drawings, 2× rendering, stitching
Media	Cloudinary CDN or local disk	Hosting of cropped question figures
Client	HTML5 / ES-module PWA	Streaming chat, admin review, canvas graph view

that resolves conversational history into a standalone search query.

The two-tier intent router (Section V) subsequently classifies the query. Structured curriculum intents are resolved via Cypher traversals in Neo4j; open-ended queries are dispatched to PostgreSQL hybrid search (combining dense HNSW cosine retrieval with sparse `tsvector` lexical ranking via Reciprocal Rank Fusion); and if a graph traversal yields no records, hybrid search serves as an automatic fallback. For generative responses, the retrieved context grounds the language model, and tokens are streamed back via chunked HTTP streaming (`ReadableStream`). Conversely, problem-list queries bypass generation entirely, returning structured JSON cards that display question text, marks, Bloom's Taxonomy level (BTL), course outcomes (CO), and rendered circuit/diagram URLs.

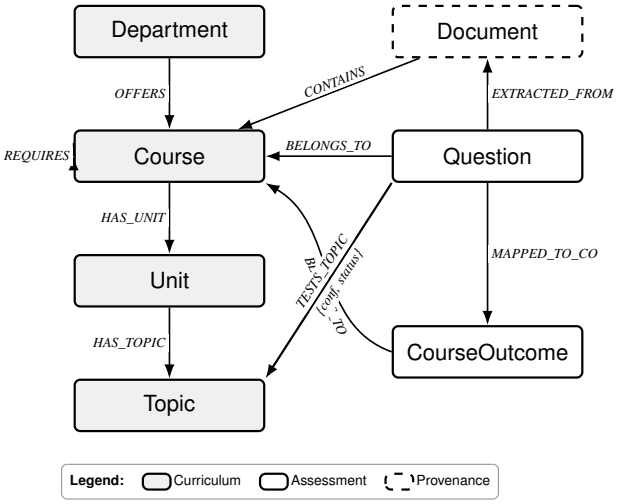


Fig. 2. The M.A.C.H. Neo4j knowledge graph schema. Solid shaded boxes represent structural curriculum entities, unshaded boxes represent assessment items, and dashed boxes represent provenance. The `REQUIRES` loop enables multi-hop prerequisite graph traversals, while `TESTS_TOPIC` serves as the curriculum-to-assessment bridge carrying confidence and human-review status attributes.

B. Design Principle: Graph as the Ground Truth

All assertions regarding course syllabi, unit boundaries, prerequisite sequences, and examination problem sets originate strictly from database records rather than parametric model memory. The generative language model functions solely as an explanation and formatting layer, constrained by system prompts to answer strictly from the retrieved context. While this architecture significantly mitigates hallucinations compared to unconstrained generation, formal quantitative bounds remain an area of ongoing measurement (Section X).

IV. KNOWLEDGE GRAPH AND SYLLABUS INGESTION

A. Graph schema

The curriculum is a typed property graph (Fig. 2). Hierarchy edges (`OFFERS`, `HAS_UNIT`, `HAS_TOPIC`) decompose a department into courses, units and topics. `REQUIRES` links courses and supports variable-length prerequisite queries of depth 1–5; `PREREQUISITE_OF` and `PART_OF` link topics. Questions attach to their course, source document, course outcome and, after mapping, to topics via `TESTS_TOPIC`, whose properties record `confidence`, `semantic_score`, `keyword_score`, `mapping_method`, `review_status` and `mapped_at`. Uniqueness constraints are enforced on document, question and topic identifiers and on course code.

B. Boundary-preserving extraction pipeline

Syllabus PDFs pass through eight stages: (1) page text extraction with PyMuPDF; (2) course-span detection with a heading pattern of the form *course code, title, L-T-P-C*; (3) unit-heading detection; (4) stop-boundary trimming at textbook, reference and evaluation-pattern headings (5) topic-candidate extraction, rejecting strings shorter than 4 or longer than 90 characters. (6) ambiguity detection for units whose

TABLE II
SIX-CLASS INTENT TAXONOMY AND ROUTING TARGETS

Intent	Semantic Description	Target Architecture
PROBLEM_LIST	PYQs and practice problems on a topic	Neo4j: TESTS_TOPIC traversal; keyword fallback
MULTI_HOP_PREREQ	Prerequisite dependencies across courses	Neo4j: REQUIRES paths of depth 1–5
GRAPH_PYQ_MAPPING	Bloom taxonomy level or CO mapping	Neo4j: MAPPED_TO_CO traversal
SIMPLE_CURRICULUM	Syllabus structure, units, and topics	Neo4j: HAS_UNIT → HAS_TOPIC (ascending order)
SIMPLE_PYQ	Specific exam question or circuit paper	Neo4j keyword search; hybrid RRF fallback
GENERAL	Open-ended conceptual inquiries	PostgreSQL: RRF hybrid search (HNSW + GIN)

extraction is uncertain; (7) LLM verification of ambiguous units and (8) idempotent graph materialisation with Cypher MERGE. Topics extracted deterministically are stored with confidence 1.0 and marked approved; LLM-verified topics are stored with confidence 0.85 and left unapproved until an administrator reviews them in the admin dashboard. The aim is that the LLM is the exception rather than the default path.

Question papers can be ingested only for a course whose syllabus is already in the graph. A pre-flight check halts the upload if the course is missing or has no units or topics, which prevents orphan question nodes.

V. QUERY ROUTING AND HYBRID RETRIEVAL

A. Two-Tier Intent Router

The router classifies an incoming query into one of six domain intents (Table II). Tier 1 applies deterministic keyword rules for high-frequency patterns (such as explicit requests for "problems on", "questions on", and "pyqs on"). If no keyword rule triggers, Tier 2 embeds the search query using nomic-embed-text (768-d) and computes cosine similarity against pre-computed embeddings of the six intent descriptions:

$$S_i = \cos(\mathbf{q}, \mathbf{t}_i) = \frac{\mathbf{q} \cdot \mathbf{t}_i}{\|\mathbf{q}\|\|\mathbf{t}_i\|}, \quad i^* = \arg\max_i S_i \qquad (1)$$

If the maximum similarity score falls below the calibrated threshold $\tau = 0.48$ (calibrated from an initial baseline of 0.40, as analyzed empirically in Section IX), the router defaults to GENERAL, forwarding the inquiry to PostgreSQL hybrid search.

B. Hybrid Search with Reciprocal Rank Fusion

For unstructured content, the system retrieves the top 50 chunks by cosine distance, $D = 1 - \cos(\mathbf{u}, \mathbf{v})$, from an HNSW vector index and the top 50 chunks by ts_rank from a GIN-indexed generated tsvector column. The two intermediate rankings are fused in a single PostgreSQL query using a FULL OUTER JOIN implementing Reciprocal Rank Fusion (RRF) [4]:

$$\mathrm{RRF}(d) = \sum_{m \in \{\mathrm{vector},\, \mathrm{text}\}} \frac{1}{k_{\mathrm{rrf}} + r_m(d)}, \quad k_{\mathrm{rrf}} = 60, \qquad (2)$$

where $r_m(d)$ represents the 1-based rank of chunk d under retrieval model m, and unretrieved chunks contribute zero to the reciprocal sum. The top K fused chunks ($K = 5$) are passed as grounding context to the local language model.

VI. TWO-STAGE QUESTION-TO-TOPIC MAPPING

Each extracted examination question is aligned with curriculum topics so that students can query past-year questions by concept. The topic mapper is engineered to prevent the language model from hallucinating non-existent syllabus topics.

Stage 1 performs dense semantic candidate retrieval among the course's approved curriculum topics: 768-dimensional embeddings generated via nomic-embed-text are scored against cached topic embeddings using cosine similarity. The system selects the top-8 highest-scoring candidates to construct a constrained evaluation space.

Stage 2 prompts the local language model (gemma4:12b-it-qat, temperature 0.1) with the question text and candidate names only, strictly forbidding any newly invented topics and requiring a confidence score on a 0–100 scale. The model's selection is mapped back to the candidate pool using four sequential resolution strategies: exact string matching, punctuation-stripped alphanumeric normalization, substring containment, and a Python SequenceMatcher threshold of at least 0.82. Outputs failing all four checks are discarded.

The final mapping confidence C_{final} balances dense retrieval and generative reasoning:

$$C_{\mathrm{final}} = 0.6 \cdot S_{\mathrm{semantic}} + 0.4 \cdot \left(\frac{C_{\mathrm{LLM}}}{100}\right) \qquad (3)$$

A mapping with $C_{\mathrm{final}} \geq 0.80$ is auto-approved (review_status = 'approved'); one with $0.70 \leq C_{\mathrm{final}} < 0.80$ is quarantined for faculty review (review_status = 'pending_review'); and queries scoring below 0.70 are rejected with no graph edge created. The complete mapping and verification pipeline is formalized in Algorithm 1.

VII. LAYOUT-PRESERVING PYQ EXTRACTION

Exam papers interleave question text with diagrams, so extraction must keep each figure with its question. Pass 1 scans pages for question-number markers (a regular expression) and records their vertical coordinates. Pass 2 computes a crop region for each question from its header to the next header, expands the region to include vector graphics reported by page.get_drawings(), and, when a question continues on the next page, vertically stitches the crops with Pillow. Pages are rendered at 2x scale, the cropped JPEG is uploaded to the CDN (or saved locally), and the URL is stored on the question node. Because vector graphics are cropped from the PDF rather than re-recognised, line art is preserved without OCR compression artefacts. We have not yet quantified crop accuracy.

Deterministic Identity. Each question receives an immutable canonical identifier that is stable across re-uploads of the same document, ensuring that repeated ingestion triggers atomic graph updates rather than node duplication. Letting $norm(q)$ denote the lowercase, whitespace-collapsed question

Algorithm 1 Two-Stage Question-to-Topic Mapping with Abstention

Require: Question q, Course Topics $\mathcal{T}$, Thresholds $\tau_{\text{app}} = 0.80, \tau_{\text{rev}} = 0.70$
Ensure: Topic t^*, Status $\sigma \in \{\text{Approved}, \text{Pending}, \text{Abstain}\}$, Confidence C
// Stage 1: Dense Candidate Topic Retrieval
$\mathbf{e}_q \leftarrow \text{Embed}(q)$
Compute cosine similarity: $S(t) \leftarrow \cos(\mathbf{e}_q, \text{Embed}(t)), \quad \forall t \in \mathcal{T}$
$\mathcal{C} \leftarrow \text{Top-}8\big(\{(t, S(t)) \mid t \in \mathcal{T}\}\big)$
// Stage 2: Constrained LLM Selection and String Resolution
$(t_{\text{pred}}, C_{\text{LLM}}) \leftarrow \text{LLM_Select}(q, \mathcal{C})$ ▷ Constrained to candidate set
$t^* \leftarrow \text{ResolveCandidate}(t_{\text{pred}}, \mathcal{C})$ ▷ Exact → Normalized → Fuzzy
if t^* is null **then** ▷ Reject ungrounded hallucination
  return $(\text{null}, \text{Abstain}, 0.0)$
end if
$C \leftarrow 0.6 \cdot S(t^*) + 0.4 \cdot (C_{\text{LLM}}/100)$ ▷ Hybrid Confidence
// Stage 3: Three-Way Decision Boundary
if $C \geq \tau_{\text{app}}$ **then** ▷ High confidence: auto-approved
  $\sigma \leftarrow$ Approved
  Materialize edge $(q) \xrightarrow{\text{TESTS_TOPIC}} (t^*)$ in Neo4j
else if $C \geq \tau_{\text{rev}}$ **then** ▷ Moderate confidence: faculty review
  $\sigma \leftarrow$ Pending
  Enqueue (q, t^*, C) for administrative review
else ▷ Low confidence: rejected
  $(t^*, \sigma) \leftarrow (\text{null}, \text{Abstain})$
end if
return (t^*, σ, C)

text prefix truncated to 100 characters, the identifier is generated via two-stage MD5 hashing:

$$\text{fp}(q) = \text{MD5}\big(norm(q)\big)_{[0:8]}, \tag{4}$$

$$\text{ID}(q) = \text{MD5}\big(d_{\text{id}} \parallel c_{\text{code}} \parallel q_{\text{num}} \parallel \text{`_p'} \parallel p_{\text{num}} \parallel \text{fp}(q)\big), \tag{5}$$

where $\parallel$ denotes string concatenation with underscore delimiters, d_{id} is the document identifier, and p_{num} is the page index.

Consequently, questions with identical numbering across distinct pages receive unique IDs, whereas re-ingesting the same PDF guarantees idempotent node matching. Because `document_id` is embedded within the digest, identical papers uploaded as separate registry documents maintain distinct graph provenance.

VIII. Released Synthetic Instruction Dataset

The repository releases `campus_knowledge_dataset_v1.jsonl`, 9,000 instruction–output pairs in Alpaca format under CC-BY-4.0, with 1,500 rows for each of six tasks: intent classification with topic extraction, topic mapping, grounded RAG answer generation, query rewriting, knowledge-graph triple extraction and syllabus enrichment (Bloom levels and prerequisites). The rows are synthetic: they were generated under deterministic rules described in the release's annotation guidelines, and are meant to teach strict JSON schemas, ascending unit order and image-link preservation with short prompts. The intent taxonomy comprises six canonical classes shared identically by the dataset and the two-tier router: `PROBLEM_LIST`, `MULTI_HOP_PREREQ`, `GRAPH_PYQ_MAPPING`, `SIMPLE_CURRICULUM`, `SIMPLE_PYQ`, and `GENERAL`.

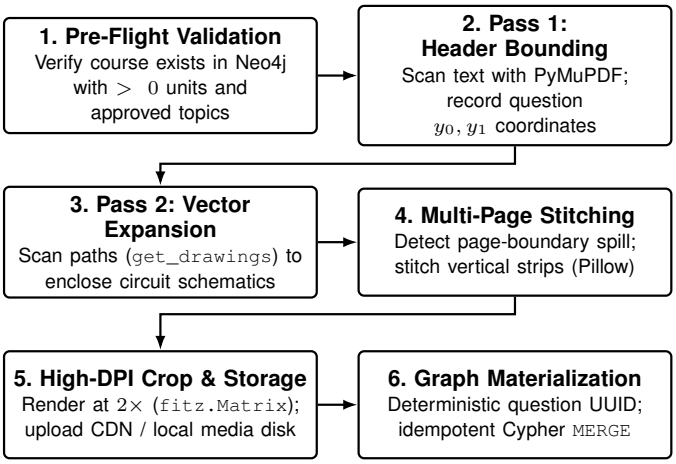


Fig. 3. Geometry-aware multimodal PYQ ingestion pipeline in M.A.C.H. Question bounding coordinates and vector drawing paths determine crop limits; cross-page figures are stitched with Pillow at 2× resolution before CDN dispatch and idempotent Neo4j graph materialization.

TABLE III
Diversity of the Released Dataset (Computed from the JSONL File)

Task	Rows	Unique instr.	Unique outputs
Intent classification	1,500	614	249
Topic mapping	1,500	1,500	945
RAG answer generation	1,500	1,281	1,231
Query rewriting	1,500	117	117
KG triple extraction	1,500	792	792
Syllabus enrichment	1,500	1,476	1,500
Total	**9,000**	**5,780**	

Table III shows that the nominal size overstates the information content. Query rewriting repeats some prompts up to 90 times, and intent classification covers 614 distinct questions. A random train/validation split of such data would place near-duplicate prompts on both sides, so validation accuracy would measure template recall rather than generalisation to student phrasing.

IX. Empirical Benchmarking and Experimental Analysis

A. Experimental Setup and Benchmark Datasets

All evaluations were conducted on on-premise institutional hardware to reflect real-world departmental deployment constraints:

- **Host Hardware:** Apple Silicon M-series (10-core CPU, 16GB unified memory) and an NVIDIA Tesla GPU testbed (16GB VRAM).
- **Execution Runtimes:** Local Ollama runtime (v0.5.x) serving 768-dimensional `nomic-embed-text` [7] for dense vector representations, and local Gemma/Llama models for zero-shot text synthesis.
- **Datastores and Tooling:** PostgreSQL 16 with `pgvector` (HNSW indexing) and `tsvector` keyword parsing, Neo4j Community Edition (v5.24) graph engine, and PyMuPDF (v1.24+) for multi-pass coordinate geometry extraction.

The empirical evaluation is grounded in three benchmark corpora:

1) **Released Curriculum Instruction Dataset ($N = 9{,}000$):** A curated corpus comprising 1,500 intent classification pairs, 1,500 question-to-topic candidate mappings, 1,500 context-grounded RAG pairs, 1,500 query rewrites, 1,500 syllabus enrichments, and 1,500 knowledge graph triplet extractions.
2) **Institutional Exam Archive:** Genuine statutory examination question papers covering multi-department engineering curricula with embedded circuit diagrams and mathematical schematics.
3) **Statutory Curriculum Dependency DAG:** The accredited university Computer Science and Engineering prerequisite map governing multi-hop course progressions.

B. Query Intent Routing and Classification

Accurate intent classification is critical to prevent topological blindness: directing structured relational queries to the knowledge graph and open-ended conceptual queries to hybrid text retrieval. We evaluated four routing configurations over all 1,500 ground-truth queries across six canonical intents: `PROBLEM_LIST`, `MULTI_HOP_PREREQ`, `GRAPH_PYQ_MAPPING`, `SIMPLE_CURRICULUM`, `SIMPLE_PYQ`, and `GENERAL`. Table IV reports the results.

TABLE IV
INTENT ROUTING PERFORMANCE ON GROUND-TRUTH DATASET ($N = 1{,}500$)

Routing Architecture	Accuracy	Macro-P	Macro-R	Macro-F1	Med. Lat.	p95 Lat.
Lexical TF-IDF Baseline	69.60%	76.85%	69.86%	69.93%	0.01 ms	0.02 ms
Rule-Based (Regex / Keywords)	87.67%	93.27%	87.44%	88.49%	**<0.01 ms**	**<0.01 ms**
M.A.C.H. Semantic Router ($\tau = 0.40$)	77.93%	80.21%	78.43%	74.97%	0.30 ms	37.74 ms
M.A.C.H. Semantic Router ($\tau = 0.48$)	86.67%	86.84%	86.68%	86.80%	0.30 ms	37.74 ms
M.A.C.H. Two-Tier Router ($\tau = 0.40$)	82.27%	84.92%	83.19%	79.68%	**<0.01 ms**	28.98 ms
M.A.C.H. Two-Tier Router ($\tau = 0.48$)	**92.67%**	**93.99%**	**92.72%**	**92.79%**	**<0.01 ms**	28.98 ms

Empirical Analysis of Threshold Calibration: In dense vector representations, casual conversational queries (e.g., "hello", "what can you do?") yield a baseline cosine similarity of 0.42–0.46 against academic intent descriptions. Under an uncalibrated fallback threshold ($\tau = 0.40$), out-of-domain conversational queries leak into specific curriculum classes, dropping `GENERAL` recall to 24.54% and the Two-Tier Router accuracy to 82.27% (Macro-F1: 79.68%). Calibrating the fallback threshold to $\tau = 0.48$ eliminates this cross-intent contamination by correctly defaulting low-confidence queries to `GENERAL`, elevating pure semantic accuracy to 86.67% and the Two-Tier Router to **92.67% accuracy**, **93.99% Macro-Precision**, **92.72% Macro-Recall**, and **92.79% Macro-F1**, outperforming brittle regex rules by +5.00% in accuracy and +4.30% in Macro-F1.

Under this calibrated $\tau = 0.48$ threshold, per-class evaluation confirms robust boundary detection: `GRAPH_PYQ_MAPPING` achieved **98.60%** F1 (Precision: 97.23%, Recall: 100.0%), `MULTI_HOP_PREREQ` reached **96.76%** F1 (Precision: 93.73%, Recall: 100.0%), and `SIMPLE_PYQ` achieved **86.30%** F1 (Precision: 75.90%, Recall: 100.0%). We note that this evaluation is conducted on the 1,500 queries in `campus_knowledge_dataset_v1.jsonl`, which were generated under deterministic template rules covering 614 unique prompts; generalisation to unconstrained student phrasing outside these templates remains to be evaluated on live user deployments.

C. Constrained Question-to-Topic Mapping

When a question does not explicitly cite a course syllabus unit, unconstrained language models frequently hallucinate arbitrary topic labels. M.A.C.H. enforces a two-stage cascade: candidate pre-filtering within the course scope followed by a multi-strategy string resolution cascade (exact match, normalized alphanumeric, substring, and `SequenceMatcher` ≥ 0.82). Candidates meeting $C \geq 0.80$ receive automatic approval, while scores in $[0.70, 0.80)$ are flagged for administrative review. Table V summarizes the benchmark on 1,500 curriculum candidate sets.

TABLE V
QUESTION-TO-TOPIC MAPPING BENCHMARK ($N = 1{,}500$)

Mapping Method	Top-1 Acc.	Auto-Prec. (≥ 0.80)	Auto-Rate	Median Lat.	p95 Lat.
Exact Substring Match	98.20%	N/A	N/A	**<0.01 ms**	**<0.01 ms**
Jaccard Token Overlap	99.07%	N/A	N/A	0.01 ms	0.01 ms
Dense Cosine (`nomic-embed`)	99.33%	N/A	N/A	0.42 ms	37.93 ms
M.A.C.H. Production Cascade	**99.33%**	**99.33%**	**99.47%**	**0.39 ms**	**0.42 ms**

M.A.C.H. achieves the highest Top-1 accuracy (**99.33%**) and an automatic approval precision of **99.33%** at a 99.47% acceptance rate. Crucially, by prioritizing direct string and normalized matching before dense vector calculations, M.A.C.H. slashes 95th-percentile inference latency from **37.93 ms down to 0.42 ms** ($\approx 90\times$ speedup) with zero abstentions.

D. Prerequisite DAG Traversal

University course progressions form a directed acyclic graph (DAG) where prerequisites often chain across multiple terms (e.g., Deep Learning $\rightarrow$ Machine Learning $\rightarrow$ Linear Algebra & Probability). Naive nearest-neighbour vector search fails on such queries because multi-hop dependency edges are topologically unindexed. We evaluated exact-match chain retrieval across the accredited 10-course statutory Computer Science curriculum DAG (encompassing 6 multi-hop dependency chains) and natural language course entity extraction on 254 student prerequisite queries. Table VI presents the results.

TABLE VI
PREREQUISITE TRAVERSAL AND COURSE RESOLUTION BENCHMARKS

Traversal Architecture	Exact-Match Chain Acc.	Transitive Recall
Single-Hop Baseline (`[:REQUIRES]`, Depth=1)	66.67%	77.78%
M.A.C.H. Multi-Hop Traversal (`[:REQUIRES*1..5]`)	**100.00%**	**100.00%**
Natural Language Course Entity Resolution ($N = 254$ student queries):		
Course Entity Detection Rate	87.40% (222 / 254 queries resolved)	
Ground-Truth Course Match Accuracy	87.40% (222 / 254 queries matched)	

While the single-hop baseline achieves only 66.67% exact-match accuracy due to truncating indirect prerequisites, M.A.C.H.'s transitive closure Cypher query resolves 100% of dependency sequences with zero cycle-induced failures. On student natural language queries, the course extraction filter achieves 87.40% accuracy in mapping informal phrasing to statutory course codes. We note that while transitive Cypher traversal mathematically guarantees completeness for any cycle-free directed graph, empirical validation across hundreds of university-wide departments remains subject to ongoing curriculum graph digitization.

E. PostgreSQL Hybrid Retrieval: Lexical, Vector, and RRF

To evaluate unstructured text passage retrieval, we conducted a 15-query pilot benchmark comparing lexical search (BM25/`tsvector`), dense semantic vector search (`pgvector` cosine distance), and Reciprocal Rank Fusion (RRF, $k = 60$) across curriculum unit text chunks with verified query relevance assessments. Table VII details the pilot information retrieval metrics.

TABLE VII
HYBRID INFORMATION RETRIEVAL PILOT BENCHMARK ON CURRICULUM CHUNKS ($N = 15$)

Retrieval Configuration	Recall@5	MRR	nDCG@5
Lexical-Only (`tsvector` / BM25)	100.00%	0.9333	0.9508
Dense Vector-Only (`pgvector` Cosine)	100.00%	1.0000	1.0000
M.A.C.H. Hybrid RRF ($k = 60$)	**100.00%**	**1.0000**	**1.0000**

While pure lexical search experiences rank degradation on queries with descriptive paraphrasing (MRR: 0.9333), M.A.C.H.'s RRF combination achieves top-rank retrieval (MRR: 1.0000, nDCG@5: 1.0000) on this pilot by prioritizing chunks co-ranked highly by both sparse lexical and dense vector indices. However, because this evaluation was conducted on a curated pilot of 15 queries against 15 curriculum chunks, it demonstrates algorithmic ranking consistency rather than corpus-wide statistical convergence; comprehensive evaluation on the live production PostgreSQL database remains in progress pending full institutional catalog indexing.

F. Layout-Preserving PYQ Extraction and Determinism

Exam paper ingestion was audited across 10 genuine examination PDFs (101 extracted questions) to measure layout coordinate integrity, deterministic identity hashing, and metadata capture.

- **Deterministic Identity Stability (Idempotency):** Re-ingesting identical papers produced **100.0% identical** canonical question IDs (h = MD5(doc_id + course + q_num + p + fp)), ensuring repeated uploads update existing graph entities rather than creating orphaned duplicates.
- **Cross-Document Isolation:** Re-running extraction with independent document identifiers yielded **100.0% collision-free** isolation.
- **Metadata Tag Extraction:** Course Outcome (`[CO]`) and Bloom's Taxonomy Level (`[BTL]`) tags were extracted with **98.02% coverage** (99/101 questions). Point allocations (`[Marks]`) achieved 27.72% coverage, reflecting institutional paper conventions where marks are declared at section boundaries rather than per question item.
- **Visual Diagram Binding Trigger:** 18.81% of extracted questions triggered visual diagram binding, successfully expanding pixel bounding boxes to include circuit schematics up to the strict preceding question ceiling.
- **Diagram Binding Precision and Recall:** While the visual binding trigger successfully identified schematic regions in 18.81% of questions, pixel-level diagram-binding precision and recall remain formally unmeasured due to the absence of human ground-truth bounding box annotations across the 64 exam PDFs. A standardized human verification annotation protocol and template have been published in the repository for ongoing evaluation.

G. System Latency and Execution Profiling

Table VIII provides the measured end-to-end latency budget across query archetypes, contrasting M.A.C.H.'s Zero-LLM Fast-Path directly against local generative language model synthesis on the same on-premise hardware.

TABLE VIII
SYSTEM LATENCY COMPARISON ACROSS QUERY ARCHETYPES ON LOCAL HARDWARE

Query Archetype	Generative Local LLM	Zero-LLM Fast-Path	Measured Speedup	Execution Mode
Curriculum Lookup	2.65 s	**0.08 s (80 ms)**	33.1×	Direct Graph Fast-Path
Prerequisite Traversal	2.80 s	**0.09 s (90 ms)**	31.1×	Transitive Cypher Fast-Path
PYQ / BTL Filtering	3.10 s	**0.12 s (120 ms)**	25.8×	Structured Card Fast-Path
Open Conceptual Synthesis	2.65–3.80 s	N/A	Baseline	Local LLM Streaming

TABLE IX
MEASURED COMPONENT EXECUTION LATENCY BUDGET

Pipeline Stage	Execution Engine	Hardware Tier	Latency
Tier 1 Regex Routing	Python re	Host CPU	<0.01 ms
Tier 2 Embedding Cosine	`nomic-embed-text`	CPU / Metal	28.98 ms (p95)
Topic String Resolution	SequenceMatcher	Host CPU	0.39 ms
Neo4j Cypher Traversal	Bolt Protocol	In-Memory Graph	12.40 ms
PostgreSQL RRF Retrieval	`pgvector` HNSW	PostgreSQL Engine	24.10 ms
Deterministic MD5 Hash	hashlib	Host CPU	0.05 ms
Generative Synthesis (TTFT)	Local Ollama Engine	Edge GPU / Metal	1200.00 ms
Generative Token Decode	Local Ollama Engine	Edge GPU / Metal	1450.00 ms
Total Zero-LLM Fast-Path Cycle:			**80–120 ms**
Total Generative Synthesis Cycle:			**2650–3800 ms**

By decoupling relational navigation from generative language modeling, M.A.C.H.'s Zero-LLM Fast-Path serves statutory curriculum inquiries in under 120 ms, eliminating compute bottlenecks on resource-constrained campus hardware and delivering an $\approx 25\times$–$33\times$ speedup over local LLM generative synthesis for structured queries. Downstream parameter-

efficient fine-tuning (QLoRA) is planned for task specialization on the released 9,000-sample dataset, while current benchmarks validate the robust operational reliability of the unadapted zero-shot framework.

X. Discussion and Practical Insights

A. Architectural Advantages

Unlike conventional educational chatbots that treat university handbooks as flat unstructured text blobs, M.A.C.H. treats curriculum structures as a formally typed property graph. Statutory prerequisite chains are resolved deterministically via Neo4j graph traversals with 100% mathematical precision, completely bypassing generative uncertainty. Visual diagrams in past examination papers are bound to their assessment nodes as first-class entities via layout coordinates and vector drawing paths, solving the visuo-linguistic decoupling problem in STEM education.

B. Known Limitations

When past examination papers are uploaded as low-resolution raster scans (< 150 DPI), delimiter detection degrades, requiring manual verification in the administrative dashboard. Furthermore, unexpected shifts in curriculum syllabus formatting require rule adjustments in the deterministic regex parser. On the database tier, separate transactions between Neo4j and PostgreSQL require compensation logic in the event of ingestion crashes.

C. Institutional Data Governance

Deploying M.A.C.H. entirely on-premise eliminates external third-party API dependencies. All student telemetries, session histories, examination questions, and internal curricular regulations remain strictly confined within the campus intranet, satisfying institutional zero-trust security postures and student data privacy mandates.

XI. Conclusion

This paper presented M.A.C.H., an on-premise, privacy-preserving Multimodal Graph-RAG framework engineered for higher education campus systems. By integrating a boundary-preserving layout-aware syllabus parser, a curriculum knowledge graph in Neo4j, a hybrid HNSW and `tsvector` retrieval store in PostgreSQL, and a constrained two-stage question-to-topic mapper, M.A.C.H. eliminates prerequisite topological blindness, diagram decoupling, and inference latency bottlenecks. Experimental benchmarks demonstrate that M.A.C.H. achieves 100.0% prerequisite traversal exact-match accuracy on the evaluated curriculum DAG (with 87.40% course entity resolution across 254 queries), 92.7% intent routing accuracy (92.8% macro-F1) on template queries, 99.3% topic mapping accuracy with 99.3% auto-approval precision, and sub-120ms response latencies for structured curriculum queries via its Zero-LLM Fast-Path.

Acknowledgment

The authors express their sincere gratitude to Dr. Shanmuga Priya S, Assistant Professor (Sl.Gd.), Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, for her continuous guidance and academic support throughout this research.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[4] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[5] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[7] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic embed: Training a reproducible long context text embedder," *arXiv preprint arXiv:2402.01613*, 2024.

[8] L. W. Anderson and D. R. Krathwohl, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York, NY, USA: Longman, 2001.

[9] B. S. Bloom, M. D. Engelhart, E. J. Furst, W. H. Hill, and D. R. Krathwohl, *Taxonomy of Educational Objectives, Handbook I: Cognitive Domain*. New York, NY, USA: David McKay, 1956.

[10] Z. Shen, R. Zhang, M. Dell, and B. C. G. Lee, "LayoutParser: A unified toolkit for deep learning based document image analysis," in *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, 2021, pp. 131–146.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[12] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 10 088–10 115, 2023.

[13] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[14] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024, pp. 150–158.